

THỰC HÀNH VI XỬ LÝ – VI ĐIỀU KHIỂN

GVHD: Trần Ngọc Đức

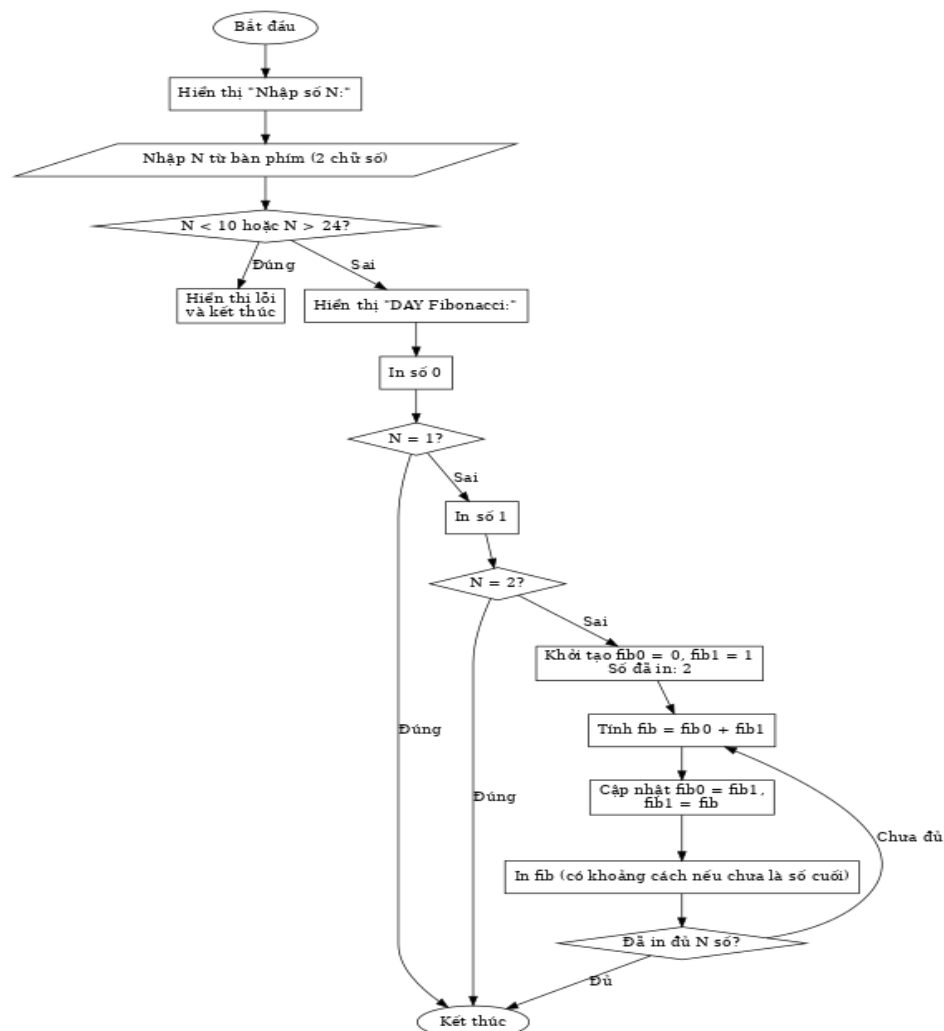
Họ và tên sinh viên thực hiện: Vũ Đại Dương

Mã số sinh viên: 23520359

BÀI THỰC HÀNH 06:

XỬ LÝ IO, TÍNH TOÁN VÀ BỘ NHỚ TRÊN 8086

I. Lưu đồ giải thuật nhập một số N có 2 chữ số từ bàn phím thông qua màn hình console và in ra màn hình n số Fibonacci đầu tiên



Hình 1 Lưu đồ thuật toán

1.Source code.

```
001 .model small
002 .stack 100h
003 .data
004     msg1 db 'Nhap so N : $'
005     msg2 db 0Dh, 0Ah, 'Day Fibonacci: $'
006     msg_error db 0Dh, 0Ah, 'So N phai tu 10 den 24! $'
007     space db ' $' ; Khoang cach giua cac so
008     n dw 0 ; Luu so N (word)
009     fibo0 dw 0 ; So Fibonacci thu 1
010     fibo1 dw 1 ; So Fibonacci thu 2
011     fibon dw 0 ; So Fibonacci tiep theo
012
013 .code
014 main proc
015     mov ax, @data
016     mov ds, ax
017
018     ; Hien thi thong bao "Nhap so N"
019     lea dx, msg1
020     mov ah, 09h
021     int 21h
022
023     ; Nhap so N (2 chu so)
024     mov ah, 01h
025     int 21h
026     sub al, '0' ; Chuyen ky tu hang chuc thanh so
027     mov bl, al ; Luu hang chuc vao bl
028
029     mov ah, 01h
030     int 21h
031     sub al, '0' ; Chuyen ky tu hang don vi thanh so
032     mov bh, al ; Luu hang don vi vao bh
033
034     ; Tinh N = hang chuc * 10 + hang don vi
035     mov al, bl
036     mov bl, 10
037     mul bl ; Nhan hang chuc voi 10 (ax = hang chuc * 10)
038     add al, bh ; Cong voi hang don vi
039     mov ah, 0 ; Dam bao phan cao cua ax = 0
040     mov n, ax ; Luu so N
041     mov cx, ax ; Su dung cx lam bo dem
042
043     ; Kiem tra N co nam trong khoang 10-24 khong
044     cmp ax, 10
045     jl invalid_input ; Neu N < 10, bao loi
046     cmp ax, 24
047     jg invalid_input ; Neu N > 24, bao loi
048     ; Hien thi thong bao "Day Fibonacci: "
049     lea dx, msg2
050     mov ah, 09h
051     int 21h
052
053     ; Kiem tra neu N = 0 thi thoat
054     cmp cx, 0 |
055     je end_program
056
057     ; In so Fibonacci dau tien (0)
058     mov ax, fibo0
059     call print_number
060
061     ; Giam bo dem
062     dec cx
```

edit: C:\emu8086\MySource\lab2.asm

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
0063      cmp cx, 0
0064      je end_program
0065
0066      ; In khoang cach
0067      lea dx, space
0068      mov ah, 09h
0069      int 21h
0070
0071      ; In so Fibonacci thu hai <1>
0072      mov ax, fibo1
0073      call print_number
0074
0075      ; Giam bo dem
0076      dec cx
0077      cmp cx, 0
0078      je end_program
0079
0080      ; In khoang cach
0081      lea dx, space
0082      mov ah, 09h
0083      int 21h
0084
0085      ; Bat dau vong lap de in cac so Fibonacci tiep theo
0086      fibo_loop:
0087      ; Tinh so Fibonacci tiep theo
0088      mov ax, fibo0
0089      mov bx, fibo1
0090      add ax, bx      ; fibn = fib0 + fib1
0091      mov fibon, ax   ; Luu so moi
0092      mov ax, fibo1
0093      mov fibo0, ax   ; Cap nhat fib0
0094      mov ax, fibon
0095      mov fibo1, ax   ; Cap nhat fib1
0096
0097      ; In so Fibonacci hien tai
0098      mov ax, fibon
0099      call print_number
0100
0101      ; In khoang cach <tru lan in so cuoi cung>
0102      cmp cx, 1
0103      je skip_space
0104      lea dx, space
0105      mov ah, 09h
0106      int 21h
0107      skip_space:
0108
0109      ; Giam bo dem va lap lai
0110      loop fibo_loop ; Giam cx va lap lai neu cx > 0
0111
0112      jmp end_program
0113
0114      invalid_input:
0115      ; Hien thi thong bao loi
0116      lea dx, msg_error
0117      mov ah, 09h
0118      int 21h
0119
0120      end_program:
0121      ; Ket thuc chuong trinh
0122      mov ah, 4Ch
0123      int 21h
0124      main endp
0125
```

line: 110

col: 85

drag a file here to open

```
edit: C:\emu8086\MySource\lab2.asm
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about

124 main endp
125
126 ; Ham in so <ho tro so nhieu chu so>
127 print_number proc
128     push ax
129     push bx
130     push cx
131     push dx
132
133     mov bx, 10
134     mov cx, 0           ; Dem so chu so
135 convert_loop:
136     mov dx, 0
137     div bx              ; Chia ax cho 10, thuong trong ax, du trong dx
138     push dx             ; Luu chu so <du> vao stack
139     inc cx              ; Tang so chu so
140     cmp ax, 0           ; Neu thuong = 0 thi dung
141     jne convert_loop
142
143 print_loop:
144     pop dx              ; Lay chu so tu stack
145     add dl, '0'         ; Chuyen thanh ky tu
146     mov ah, 02h
147     int 21h             ; In ky tu
148     loop print_loop
149
150     pop dx
151     pop cx
152     pop bx
153     pop ax
154     ret
155 print_number endp
156
157 end main
158
```

line: 110 col: 85 drag a file here to open

2. Giải thích cách thức hoạt động.

🚦 Phân khai báo dữ liệu (.data):

.model small: Sử dụng mô hình bộ nhớ nhỏ, phù hợp với chương trình đơn giản.

.stack 100h: Cấp phát 256 byte cho ngăn xếp (stack).

Khai báo biến:

msg1 db 'Nhap so N (2 chu so, 10-24): \$': Chuỗi thông báo yêu cầu nhập N.

msg2 db 0Dh, 0Ah, 'DAY Fibonacci: \$': Chuỗi tiêu đề dãy Fibonacci, với 0Dh, 0Ah để xuống dòng.

msg_error db 0Dh, 0Ah, 'So N phai tu 10 den 24! \$': Chuỗi thông báo lỗi khi N không hợp lệ.

space db ' \$': Chuỗi khoảng cách giữa các số khi in.

n dw 0: Biến lưu số N nhập vào (16-bit, dw là kiểu word).

fib0 dw 0: Số Fibonacci đầu tiên (0).

fib1 dw 1: Số Fibonacci thứ hai (1).

fibn dw 0: Số Fibonacci tiếp theo, dùng để tính toán trong vòng lặp.

Phần code chính (main proc):

1. Khởi tạo chương trình:

mov ax, @data: Lấy địa chỉ của segment dữ liệu.

mov ds, ax: Gán địa chỉ này vào thanh ghi ds (data segment) để truy cập các biến.

2. Hiển thị thông báo nhập N:

lea dx, msg1: Tải địa chỉ của chuỗi msg1 vào dx.

mov ah, 09h: Chức năng 09h của ngắt int 21h để in chuỗi.

int 21h: Thực hiện in chuỗi msg1.

Nhập số N:

Nhập hàng chục:

mov ah, 01h: Chức năng 01h của int 21h để nhập một ký tự từ bàn phím.

int 21h: Ký tự nhập vào được lưu trong al.

sub al, '0': Chuyển ký tự số (ví dụ: '1') thành giá trị số (1) bằng cách trừ mã ASCII của '0'.

mov bl, al: Lưu giá trị hàng chục vào bl.

Nhập hàng đơn vị:

Tương tự, nhập ký tự thứ hai và lưu vào bh.

- Tính N:

mov al, bl: Đưa giá trị hàng chục vào al.

mov bl, 10: Đặt bl là 10 để nhân.

mul bl: Nhân al với 10, kết quả lưu trong ax ($ax = \text{hàng chục} * 10$).

add al, bh: Cộng thêm giá trị hàng đơn vị.

mov ah, 0: Đặt phần cao của ax về 0 (đảm bảo ax là số 16-bit đúng).

mov n, ax: Lưu N vào biến n.

mov cx, ax: Sử dụng cx làm bộ đếm vòng lặp sau này.

Kiểm tra N hợp lệ:

cmp ax, 10: So sánh N với 10.

jl invalid_input: Nếu $N < 10$, nhảy đến nhãn invalid_input.

cmp ax, 24: So sánh N với 24.

jg invalid_input: Nếu $N > 24$, nhảy đến invalid_input.

Xử lý lỗi:

invalid_input:: In chuỗi msg_error và thoát chương trình.

Hiển thị tiêu đề dãy Fibonacci:

lea dx, msg2: Tải địa chỉ của msg2.

mov ah, 09h: Chức năng in chuỗi.

int 21h: In DAY Fibonacci:.

Kiểm tra trường hợp đặc biệt:

cmp cx, 0: Kiểm tra nếu $N = 0$.

je end_program: Nếu $N = 0$, thoát chương trình ngay lập tức.

In số Fibonacci đầu tiên (0):

mov ax, fib0: Lấy giá trị fib0 (0) vào ax.

call print_number: Gọi hàm print_number để in số 0.

dec cx: Giảm bộ đếm cx.

cmp cx, 0: Kiểm tra nếu $cx = 0$.

je end_program: Nếu $cx = 0$ (tức là $N = 1$), thoát chương trình.

In khoảng cách và số Fibonacci thứ hai (1):

lea dx, space: In khoảng cách giữa các số.

mov ah, 09h: Chức năng in chuỗi.

int 21h: In ' '.

mov ax, fib1: Lấy giá trị fib1 (1) vào ax.

call print_number: In số 1.

dec cx: Giảm bộ đếm cx.

cmp cx, 0: Kiểm tra nếu $cx = 0$.

je end_program: Nếu $cx = 0$ (tức là $N = 2$), thoát chương trình.

In khoảng cách lần nữa.

Vòng lặp tính và in các số Fibonacci tiếp theo (fib_loop):

Tính số Fibonacci tiếp theo:

mov ax, fib0: Lấy số Fibonacci trước đó (fib0).

mov bx, fib1: Lấy số Fibonacci hiện tại (fib1).

add ax, bx: Tính $fibn = fib0 + fib1$.

mov fibn, ax: Lưu kết quả vào fibn.

mov ax, fib1: Cập nhật $fib0 = fib1$.

mov fib0, ax: Lưu giá trị mới cho fib0.

mov ax, fibn: Cập nhật $fib1 = fibn$.

mov fib1, ax: Lưu giá trị mới cho fib1.

In số Fibonacci hiện tại:

mov ax, fibn: Lấy số vừa tính.

call print_number: In số đó.

In khoảng cách (trừ số cuối cùng):

cmp cx, 1: Kiểm tra nếu đây là số cuối cùng ($cx = 1$).

je skip_space: Nếu là số cuối, bỏ qua in khoảng cách.

lea dx, space: In khoảng cách nếu chưa phải số cuối.

mov ah, 09h: Chức năng in chuỗi.

int 21h: In ''.

skip_space:: Nhấn để bỏ qua in khoảng cách.

Giảm bộ đếm và lặp lại:

loop fib_loop: Giảm cx và nhảy lại nhãn fib_loop nếu cx > 0.

Kết thúc chương trình:

jmp end_program: Nhảy đến nhãn kết thúc.

end_program:: Dùng mov ah, 4Ch và int 21h để thoát chương trình và trả quyền điều khiển về hệ điều hành.

Hàm print_number:

Mục đích: Chuyển đổi một số trong ax thành chuỗi ký tự và in ra màn hình.

Chi tiết:

Lưu các thanh ghi: push ax, push bx, push cx, push dx để bảo vệ giá trị hiện tại của các thanh ghi.

Chuyển đổi số thành chữ số:

mov bx, 10: Sử dụng 10 làm cơ số để chia lấy nguyên và dư.

mov cx, 0: Khởi tạo bộ đếm số chữ số.

convert_loop:: Vòng lặp chuyển đổi.

mov dx, 0: Đặt dx về 0 để chia 16-bit (dx:ax chia cho bx).

div bx: Chia ax cho 10, thương lưu trong ax, dư lưu trong dx.

push dx: Lưu chữ số (dư) vào stack.

inc cx: Tăng bộ đếm số chữ số.

cmp ax, 0: Kiểm tra nếu thương = 0 (hết số để chia).

jne convert_loop: Nếu chưa hết, lặp lại.

In các chữ số:

print_loop:: Vòng lặp in.

pop dx: Lấy chữ số từ stack.

add dl, '0': Chuyển số thành ký tự ASCII (ví dụ: 5 → '5').

mov ah, 02h: Chức năng 02h của int 21h để in một ký tự.

int 21h: In ký tự.

loop print_loop: Lặp lại cho đến khi in hết (cx về 0).

Khôi phục thanh ghi và kết thúc:

pop dx, pop cx, pop bx, pop ax: Khôi phục giá trị các thanh ghi.

ret: Trả về từ hàm.

3. Testcase

✚ Testcase 1:

```
SCR emulator screen (80x25 chars)
Nhap so N: 10
Day Fibonacci: 0 1 1 2 3 5 8 13 21 34
```

✚ Testcase 2:

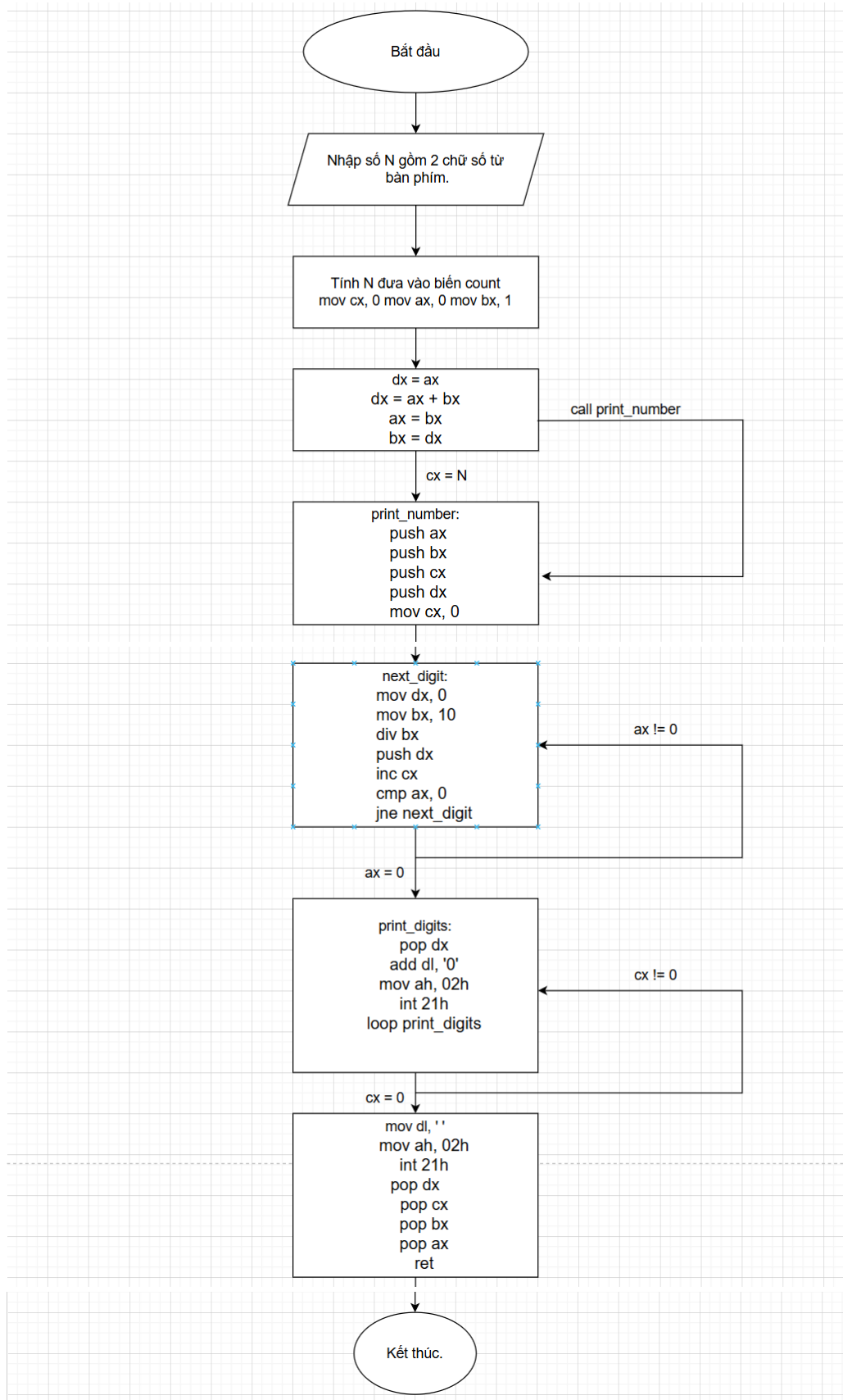
```
SCR emulator screen (80x25 chars)
Nhap so N: 15
Day Fibonacci: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

✚ Testcase 3:

```
SCR emulator screen (80x25 chars)
Nhap so N: 24
Day Fibonacci: 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657
```

II. Lưu đồ thuật toán xử lý và cách làm khác của yêu cầu trên:

1. Lưu đồ thuật toán



2.Source code

```

001 .model small
002 .stack 100h
003 .data
004     count db 0
005     msg db 'Day fibo: $'
006 .code
007 main:
008     mov ax, @data
009     mov ds, ax
010
011     mov ah, 01h
012     int 21h
013     sub al, '0'
014     mov bl, al
015
016     mov ah, 01h
017     int 21h
018     sub al, '0'
019     mov bh, al
020
021
022     mov al, bl
023     mov cl, 10
024     mul cl
025     add al, bh
026     mov count, al
027
028     mov ah, 02h
029     mov dl, 13
030     int 21h
031     mov dl, 10
032     int 21h
033
034
035     mov ah, 09h
036     lea dx, msg
037     int 21h
038
039     mov cx, 0
040     mov ax, 0
041     mov bx, 1
042
043 print_loop:
044     push ax
045     call print_number
046     pop ax
047
048     mov dx, ax
049     add dx, bx
050     mov ax, bx
051     mov bx, dx
052
053     inc cx
054     mov dl, count
055     cmp cl, dl
056     jl print_loop
057
058     mov ah, 4ch
059     int 21h
060
061 print_number:
062     push ax
063     push bx
064     push cx
065     push dx
066
067     mov cx, 0
068 next_digit:
069     mov dx, 0
070     mov bx, 10
071     div bx
072     push dx
073     inc cx
074     cmp ax, 0
075     jne next_digit
076
077 print_digits:
078     pop dx
079     add dl, '0'
080     mov ah, 02h
081     int 21h
082     loop print_digits
083
084     mov dl, ' '
085     mov ah, 02h
086     int 21h
087
088     pop dx
089     pop cx
090     pop bx
091     pop ax
092     ret

```

3. Giải thích code, các bước xử lý trong code và trình bày kết quả thực hiện.

✚ Giải thích code:

```
.model small
.stack 100h
.data
    count db 0
    msg db 'Day fibo: $'
.code
```

- Chọn mô hình bộ nhớ nhỏ với lệnh .model small
- Cấp phát 256 byte cho ngăn xếp với lệnh .stack 100h
- Khai báo vùng dữ liệu với lệnh .data
- Lưu "Day fibo: " vào msg
- Count là biến 1 byte lưu số Fibonacci cần in.

```
main:
    mov ax, @data
    mov ds, ax
```

- Gán thanh ghi DS bằng segment của data để truy cập được biến count.

```
mov ah, 01h
int 21h
sub al, '0'
mov bl, al
```

```
mov ah, 01h
int 21h
sub al, '0'
mov bh, al
```

- AH = 01h, INT 21h: chờ người dùng gõ 1 ký tự
- Trừ '0' để chuyển ASCII sang số
- Lưu chữ số hàng chục vào BL, hàng đơn vị vào BH

```
mov ah, 02h
mov dl, 13
int 21h
mov dl, 10
int 21h
```

```
mov ah, 09h
lea dx, msg
int 21h
```

- In xuống dòng và "Day Fibo: "

```

mov al, bl
mov cl, 10
mul cl
add al, bh
mov count, al

```

```

mov cx, 0
mov ax, 0
mov bx, 1

```

- mul cl → phép nhân 8-bit $al \times cl \rightarrow$ kết quả 16-bit trong AX
- AL + BH → được số thực sự
- Lưu vào count

```

mov cx, 0
mov ax, 0
mov bx, 1

```

- Khởi tạo dãy Fibonacci

```

print_loop:
    push ax
    call print_number
    pop ax

```

- Gọi print_number để in số Fibonacci hiện tại (ax)
- Dùng push/pop để giữ giá trị ax

```

mov dx, ax
add dx, bx
mov ax, bx
mov bx, dx

```

- Cập nhật AX và BX cho số kế tiếp trong dãy Fibonacci

```

inc cx
mov dl, count
cmp cl, dl
jl print_loop

```

- So sánh CL với count → nếu chưa đủ N số → lặp lại

```
mov ah, 4ch
int 21h
```

- Dừng chương trình (INT 21h – dịch vụ thoát)

```
print_number:
    push ax
    push bx
    push cx
    push dx
```

- Thủ tục print_number – In số trong AX
- Lưu các thanh ghi đang dùng

```
next_digit:
    mov cx, 0
    mov dx, 0
    mov bx, 10
    div bx
    push dx
    inc cx
    cmp ax, 0
    jne next_digit
```

- Tách số thành chữ số thập phân
- Chia AX thành từng chữ số (tách ra từ hàng đơn vị), lưu vào stack để in theo đúng thứ tự

```
print_digits:
    pop dx
    add dl, '0'
    mov ah, 02h
    int 21h
    loop print_digits

    mov dl, ' '
    mov ah, 02h
    int 21h
```

- In từng chữ số ra khỏi stack, chuyển thành ký tự và in ra rồi sau đó in dấu cách.

```

pop dx
pop cx
pop bx
pop ax
ret

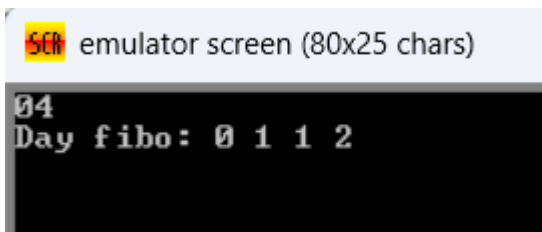
```

- Khởi phục thanh ghi và kết thúc thủ tục

4. Testcase

✚ Vì chương trình xây dựng là nhập testcase phải có 2 chữ số nên nếu muốn nhập số < 10 thì phải thêm số 0 ở đằng trước.

- Testcase 1



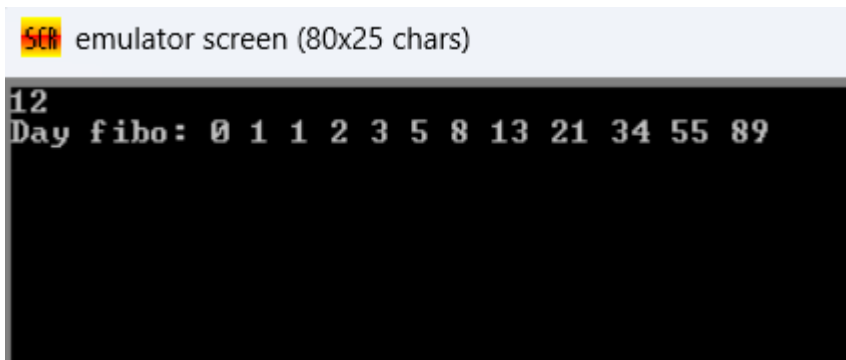
```

emulator screen (80x25 chars)
04
Day fibo: 0 1 1 2

```

- Nhập n=4 thì 4 số fibonacci đầu tiên sẽ là 0 1 1 2 => ĐÚNG

- Testcase 2



```

emulator screen (80x25 chars)
12
Day fibo: 0 1 1 2 3 5 8 13 21 34 55 89

```

- Nhập n=12 thì 12 số fibonacci đầu tiên sẽ là 0,1,1,2,3,5,8,13,21,34,55,89 => ĐÚNG